Razonamiento Automático 2026/27
Challenge 4: Soluciones de entrenamiento

Tiro al plato: push/pop, ECS muy básica, multi-fichero, gráficos y animación

> ⚠ **Cómo usar este documento**
> Intenta primero los 8 ejercicios por tu cuenta. Compara después con esta solución **y revísala críticamente**: pueden haber soluciones distintas, pero deben hacer correctamente lo que el ejercicio pide.

> ⚡ **Muy importante**
> Cada rutina de esta solución se ha verificado ensamblando y enlazando de verdad contra `engine.asm` (`rgbasm` / `rgblink`) y ejecutándola con casos de prueba. Ojo: el Ejercicio 5 da una **segunda versión** de `system_move_clay` (mismo nombre, firma distinta: la fila ya no es `ROW_CLAY` sino un parámetro). Las dos versiones no coexisten en el mismo `game.asm` a la vez: son para comparar antes/después, no para juntarlas. Usa el emulador y el depurador para comprobar que tu propia solución hace lo que debe.

Ejercicio 1: `system_move_clay`

system_move_clay

```
system_move_clay::
   ld  b, ROW_CLAY      ;; B = row, for tile_address's contract
   ld  a, [clay_col]    ;; A = current column
   ld  c, a             ;; C = column, for tile_address's contract
   call tile_address     ;; HL = address of the clay's current cell
   ld  [hl], BLANK_TILE  ;; Clear that cell

   ld  a, [clay_col]  ;; A = current column
   add a, CLAY_STEP   ;; A = next column
   ld  [clay_col], a  ;; store the updated column

   ld  b, ROW_CLAY       ;; / B = row
   ld  c, a              ;; \ C = new column   (tile_address's contract)
   call tile_address      ;; HL = address of the clay's new cell
   ld  [hl], CLAY_TILE_A  ;; Draw the clay there
   ret
```

Ideas clave:

- `ROW_CLAY` es fijo (una constante), así que `B` se recarga con el mismo valor las dos veces que se llama a `tile_address` ; lo único que cambia entre la primera y la segunda llamada es `C` (la columna).
- Se borra la celda **antes** de mover la columna, y se dibuja **después**: si se hiciera al revés, se borraría la celda nueva en vez de la vieja.
- Esta versión no gestiona el borde (`CLAY_MAX_COL`) a propósito: es una simplificación deliberada del entrenamiento, para no repetir dos veces seguidas la misma lógica de wraparound; la versión completa está en el Ejercicio 5.

Ejercicio 2: Completa `system_move_shot`

system_move_shot (completo)

```
;; Moves the shot one row upward each call.
;; When it reaches ROW_CLAY, resolves the shot: checks
;; for a hit and deactivates it either way.
system_move_shot::
   ld  a, [shot_active] ;; A = indicates if a shot is active
   or  a                ;; Is A == 0? (no shot is active)
   ret z                ;; If no shot is active, return
                        ;;   (nothing to do)

   ;; Erase the shot at its current cell
   ld  a, [shot_row]  ;; A = the shot's current row
   ld  b, a           ;; B = row, for tile_address's contract
   ld  a, [shot_col]  ;; A = the shot's current column
   ld  c, a           ;; C = column, for tile_address's contract
   call tile_address  ;; HL = address of the shot's current cell
   ld  [hl], BLANK_TILE ;; Clear the cell

   ;; Move the shot ONE ROW closer to the clay (upwards)
   ld  a, [shot_row]  ;; /
   dec a              ;; | Move shot_row 1 ROW Upwards
   ld  [shot_row], a  ;; \

   ;; Check if it reaches the CLAY ROW
   cp  ROW_CLAY       ;; has the shot reached the clay's row?
   jr  nz, .keep_flying ;; if it hasn't arrived yet, .keep_flying

.clay_row_reached
   ;; The shot reached the clay's row: find out whether it
   ;; was a hit, but only count it in `hits` when it was a hit
   call system_check_hit ;; Call the routine that checks for a hit
   jr  nc, .no_hit        ;; It was a miss (NC): skip past the count
   ld  hl, hits           ;; HL = address of the hits counter
   inc [hl]               ;; hits counter += 1
.no_hit
   ld  hl, attempts  ;; HL = address of the attempts counter
   inc [hl]          ;; every resolved shot counts as an attempt
   xor a                ;; A = 0
   ld  [shot_active], a ;; the shot is resolved either way
   ret

.keep_flying
   ;; Draw the shot at its new row
   ld  b, a           ;; B = new row (A still holds the new row)
   ld  a, [shot_col]  ;; / C = shot's column
   ld  c, a           ;; \   (for tile_address's contract)
   call tile_address  ;; HL = address of the shot's new cell
   ld  [hl], SHOT_TILE ;; Draw the shot
   ret
```

Ideas clave (hueco a hueco):

- `ret z`: tras `or a`, `Z` se activa justo cuando `A` (`shot_active`) vale 0. Queremos volver pronto cuando **no** hay disparo en vuelo, así que la condición es `z`, no `nz`.
- `ld [hl], BLANK_TILE`: el comentario "clear that cell" es literal aquí: una vez `tile_address` deja la dirección en `hl`, borrar es escribir `BLANK_TILE`.

- `dec a`: el comentario da la **dirección** del cambio (fila 12→fila 5, el número baja), no el nombre de la instrucción: obliga a conectar "baja" con `dec`, no a copiar una palabra.
- `jr nz, .keep_flying`: `cp ROW_CLAY` deja `Z` activo cuando la fila **ya ha llegado**. "Sigue en vuelo" es el caso contrario, así que `nz`.
- El bloque de 4 líneas (`call system_check_hit` / `jr nc, .no_hit` / `ld hl, hits` / `inc [hl]`) es el hueco "interesante": no es una instrucción suelta, es una decisión completa. Hay que llamar a la comprobación, saltar fuera si falla, y solo entonces incrementar `hits`, en ese orden exacto, o un fallo contaría como impacto.
- `ld [hl], SHOT_TILE`: igual que el hueco de borrar, en espejo: ahora se dibuja en vez de borrar.

> ⚡ **Error típico**
> En el bloque de 4 líneas, incrementar `hits` **antes** de comprobar el flag (o sin el `jr nc` de por medio): así todo disparo contaría como impacto, acertase o no.

Ejercicio 3: Reescribe `system_check_hit`

system_check_hit (reescrito)

```
;; 📥 shot_col, clay_col in WRAM. 🔙 Carry: C=hit, NC=miss
system_check_hit::
   ld  a, [shot_col]  ;; / B = shot_col
   ld  b, a           ;; \
   ld  a, [clay_col]  ;; / C = clay_col
   ld  c, a           ;; \
   ld  a, b           ;; A = shot_col again
   sub c              ;; A = shot_col - clay_col (may end up negative)
   jr  nc, .positive
   cpl                ;; it went negative (there was a carry): negate it
   inc a
.positive             ;; A = |shot_col - clay_col|
   cp  HIT_TOLERANCE + 1 ;; carry if A <= HIT_TOLERANCE: hit
   ret
```

Ideas clave:

- Una única resta (`sub c`) basta: el signo del resultado (flag `C`) dice si `shot_col < clay_col` o al revés; negar con `cpl` + `inc a` da el valor absoluto de la diferencia en cualquier caso.
- `cp HIT_TOLERANCE + 1` activa el **carry** justo cuando la diferencia es `<= HIT_TOLERANCE` (`A < HIT_TOLERANCE + 1`). Como el contrato devuelve el resultado en el carry, basta con `ret`: no hay que forzar ningún flag. Por eso el carry es el flag más práctico para esta rutina.
- Nunca hace falta `hl`: los dos valores a comparar están en direcciones fijas de WRAM, cargables directamente con `ld a, [shot_col]` (la forma corta, válida solo para `A`).

> ⚡ **Error típico**
> Comparar con `cp HIT_TOLERANCE` en vez de `cp HIT_TOLERANCE + 1`. `cp X` activa el carry cuando `A < X`, así que con `cp HIT_TOLERANCE` una diferencia de exactamente `HIT_TOLERANCE` (1) saldría como fallo, y debe contar como impacto.

Ejercicio 4: `system_draw_hud`

system_draw_hud

```
system_draw_hud::
   ld  hl, HUD_ATTEMPTS_CELL ;; HL = where to draw
   push hl                   ;; save it: digit_to_tile destroys HL
   ld  a, [attempts]         ;; A = the digit to draw
   call digit_to_tile        ;; A = the tile for that digit
   pop  hl                   ;; recover the destination address
   ld  [hl], a               ;; draw it
   ret
```

Ideas clave:

- `hl` se calcula **antes** de la llamada (es la dirección donde hay que escribir), pero hace falta **después** (para escribir ahí), y `digit_to_tile` lo destruye por dentro. `push hl` / `pop hl` alrededor de la llamada son exactamente "guarda antes, recupera después".
- El orden importa: `push hl` va **antes** de `call digit_to_tile` (para guardar el valor bueno), y `pop hl` va **después** (para recuperarlo ya destruido). Si se intercambiaran, se guardaría/recuperaría basura.
- `digit_to_tile` recibe y devuelve en `A`: no hace falta guardar `A`, solo lo que la llamada destruye y tú necesitas después (`hl`).

> **Error típico**
> Guardar `hl` con `push hl` pero no hacer `pop hl` después (o hacerlo con otro registro, p. ej. `pop de`). La pila queda descuadrada: la próxima vez que la rutina haga `ret`, no encontrará la dirección de retorno correcta.

Ejercicio 5: De constante a parámetro

system_move_clay (fila por parámetro)

```
;; 📥 INPUT: A = row the clay flies on (used to be the constant ROW_CLAY)
system_move_clay::
   push af              ;; save the row: tile_address destroys everything

   ld  b, a             ;; B = row, for tile_address's contract
   ld  a, [clay_col]    ;; A = current column
   ld  c, a             ;; C = column, for tile_address's contract
   call tile_address     ;; HL = address of the clay's current cell
   ld  [hl], BLANK_TILE  ;; Clear that cell

   ld  a, [clay_col]    ;; A = current column
   add a, CLAY_STEP     ;; A = next column
   cp  CLAY_MAX_COL + 1 ;; did it go past the edge?
   jr  nz, .no_wrap
   ld  a, CLAY_MIN_COL  ;; yes: wrap back to the start
.no_wrap
   ld  [clay_col], a    ;; store the updated column
   ld  c, a             ;; C = new column

   pop  af               ;; recover the row
   ld  b, a              ;; B = row, for tile_address's contract
   call tile_address      ;; HL = address of the clay's new cell
   ld  [hl], CLAY_TILE_A  ;; Draw the clay there
   ret
```

Ideas clave:

- Según su contrato, `tile_address` puede modificar `A`, `BC` y `DE`, así que la fila que llega por parámetro (en `A`) no tiene garantizado sobrevivir a la primera llamada. Aunque el código actual del motor no toque `B`, el contrato no lo promete: si confiamos en un detalle de implementación, el día que el motor cambie, nuestro código deja de funcionar. Se guarda con `push af` al entrar y se recupera con `pop af` justo antes de la segunda llamada: exactamente el mismo patrón del Ejercicio 4 (guardar antes de una llamada que destruye, recuperar después), aplicado ahora a un parámetro de entrada en vez de a un puntero calculado.
- Entre el `push af` y el `pop af` es libre usar `A`, `B`, `C` para lo que haga falta (aquí, calcular la nueva columna): lo importante ya está a salvo en la pila.
- `CLAY_MIN_COL`, `CLAY_MAX_COL` y `CLAY_STEP` siguen siendo constantes `DEF`: no todo tiene que convertirse en parámetro a la vez. Aquí solo cambia lo que de verdad interesa poder variar (la altura).

> **Error típico**
> Hacer `push af` pero recuperar con `pop bc` (o cualquier registro que no sea `af`): la pila queda igual de descuadrada que con un `push` / `pop` de punteros. La pareja tiene que coincidir en registro, aunque el contenido que te interesa (la fila) solo esté en la mitad alta de `af` (el registro `a`).

> **Reflexión: para qué sirve**
> Sin este cambio, tener platos a distintas alturas en distintos niveles significaría tener una copia de `system_move_clay` por cada altura (o editar `ROW_CLAY` y volver a ensamblar cada vez); el copy-paste-y-cambia-una-constante de siempre. Con la fila como parámetro, **la misma rutina** sirve para cualquier altura: quien la llama decide, sin tocar la rutina ni recompilar. Es el primer paso hacia lo que harás en el Challenge 5 con estructuras completas: en vez de pasar un solo valor por parámetro, pasarás **varios datos relacionados a la vez** mediante un puntero a una mini-estructura.

Ejercicio 6: Multi-fichero

6.1. `tile_address:` con un solo `:` **ensambla** `engine.asm` **sin ningún problema** (es una etiqueta perfectamente válida dentro de ese fichero). El fallo aparece **al enlazar** (`rgblink`), cuando intenta resolver el `call tile_address` de `game.o` contra los símbolos que `engine.o` exporta: como `tile_address` no está exportada , el enlazador la trata como si no existiera fuera de `engine.asm`, y da un error de "símbolo desconocido" en el `.o` de `game.asm`.

6.2. También **falla al enlazar**, no al ensamblar. Al traducir `game.asm`, `rgbasm` no conoce el valor de `TILEMAP`, pero no le hace falta para seguir: deja el hueco en el `.o` para que lo rellene el enlazador (como con una etiqueta). Al enlazar, `rgblink` busca `TILEMAP` entre los símbolos exportados, no la encuentra y da el error: *"A constant with that name is defined but not exported"*.

> **Para saber más (no se pide)**
> Solo fallaría **al ensamblar** si la constante se usara donde `rgbasm` necesita su valor *mientras traduce*: dentro de otro `DEF`, en un `IF` o en un `DS`. En instrucciones normales (`ld hl, TILEMAP`, `cp TILEMAP + 1`...) el valor puede esperar al enlazador.

Ejercicio 7: `memcpy` y `load_clay_tile`

memcpy / load_clay_tile

```
ClayTileData:
   DB %00111100, %00111100
   DB %01000010, %01000010
   DB %10000001, %10000001
   DB %10000001, %10000001
   DB %10000001, %10000001
   DB %01000010, %01000010
   DB %00111100, %00111100
   DB %00000000, %00000000

;; 📥 HL = source, DE = destination, B = number of bytes
;; Copies B bytes from [HL] onwards to [DE] onwards.
;; Reusable: works for tiles, or for copying ANY block of bytes.
memcpy::
.loop
   ld  a, [hl+]  ;; read one byte, HL advances on its own
   ld  [de], a   ;; write it to [DE]
   inc de        ;; DE doesn't auto-advance like HL+ does: move it by hand
   dec b
   jr  nz, .loop
   ret

load_clay_tile::
   ld  hl, ClayTileData              ;; HL = source: the tile data itself
   ld  de, $8000 + CLAY_TILE_A * 16  ;; DE = destination: tile CLAY_TILE_A in VRAM
   ld  b, 16                         ;; B = bytes left to copy (a tile is 16 bytes)
   call memcpy
   ret
```

Ideas clave:

- `[hl+]` avanza el origen solo; `de` (destino) hay que avanzarlo aparte con `inc de`, igual que en el Challenge 2 (solo `hl` soporta `[reg+]`).
- `memcpy` no sabe nada de tiles ni de platos: solo copia bytes de un sitio a otro. `load_clay_tile` decide QUÉ copiar (`ClayTileData`) y A DÓNDE (`CLAY_TILE_A`). Es la misma idea de "de constante a parámetro" del Ejercicio 5, aplicada esta vez a una rutina entera en vez de a un solo valor.
- 16 bytes = 8 filas × 2 bytes/fila (formato 2bpp: cada fila del tile necesita 2 bytes, uno con el bit bajo de cada píxel y otro con el bit alto). No hace falta entender el formato a fondo para esta rutina; basta saber que son 16 bytes seguidos.
- `$8000 + CLAY_TILE_A * 16` se calcula en tiempo de ensamblado (es aritmética entre constantes): el resultado es una dirección fija, no hay que calcular nada en tiempo de ejecución.

> 🔥 **Reflexión**
> Cualquier copia de un bloque de memoria: otro tile, una tabla de datos de la ROM que se quiera modificar en WRAM, una zona del tilemap que se quiera guardar para restaurarla después, etc. `memcpy` no impone nada sobre el contenido que mueve; por eso sirve para todo eso sin cambiar ni una línea.

Ejercicio 8: `system_animate_clay`

system_animate_clay

```
system_animate_clay::
   ld  hl, anim_frame ;; HL = the "which frame" flag
   ld  a, [hl]
   xor 1              ;; flip it: 0<->1
   ld  [hl], a

   ld  b, ROW_CLAY    ;; / B = row
   ld  a, [clay_col]  ;; | A = column
   ld  c, a           ;; \ C = column, for tile_address's contract
   call tile_address    ;; HL = address of the clay's cell
   ld  a, [anim_frame] ;; A = new frame (tile_address modified A)
   or  a               ;; is the new frame 0?
   jr  nz, .frame_b
   ld  [hl], CLAY_TILE_A
   ret
.frame_b
   ld  [hl], CLAY_TILE_B
   ret
```

Ideas clave:

- `xor 1` es la forma habitual de "invertir un flag de 0/1 a 1/0" en un solo byte: si `anim_frame` vale 0, `xor 1` da 1; si vale 1, da 0.
- `tile_address` modifica `A`, así que hay que **volver a leer** `anim_frame` después de la llamada para decidir qué tile dibujar. `hl` sí sigue apuntando a la celda del plato.
- Esta versión alterna en **cada** llamada, a propósito. Quien decida la cadencia (cada cuántos frames llamarla) queda fuera del ejercicio: es una decisión de `game_loop`, no de esta rutina.

> ⚡ **Error típico**
> Comprobar `A` después de `call tile_address` creyendo que aún contiene `anim_frame`: la llamada lo ha modificado (contiene la columna), así que se dibujaría siempre el mismo tile. También, dibujar antes de invertir `anim_frame`: se dibujaría el tile del frame anterior.